

Mocks avec Mockito

Pour les tests utilisant les mocks, on a décidé de tester les conditions d'une méthode dans la classe Router ([checkNoLegacyParameters](#)) qui prend en comme argument un objet GHRequest et qui lance des exceptions si certains des arguments de cette requête sont considérés obsolètes qui ne sont plus acceptés. La logique des tests sera alors de tester s'il y aura bel et bien la bonne exception qui sera lancée si une des conditions est remplie (s'il y a un paramètre « legacy » ou obsolète).

Les tests utilisent plusieurs mocks qu'il faudra initialiser afin de pouvoir appeler la méthode ... (voir la méthode pour le « setup » de notre classe de test avec le [@BeforeAll](#) et celle avec le [@BeforeEach](#)). En premier lieu, avant de faire quoi que ce soit, il faut initialiser la classe Router 10 paramètres dont 7 peuvent être des valeurs « null », mais il faut tout de même faire des mocks de la classe Map pour les arguments suivants:

- « chGraphs » (le constructeur appelle sa méthode [isEmpty\(\)](#))
- « landmarks » (le constructeur appelle sa méthode [isEmpty\(\)](#))
- « profilesByName » (le constructeur appelle sa méthode [keySet\(\)](#))

En deuxième lieu, pour le « setup » avant chaque test, il faut savoir qu'il y aura deux classes qu'il faudra nécessairement mocker. Tout d'abord, on mock la requête GHRequest qui est utilisé dans chaque condition dont on n'utilise que la méthode [getHints\(\)](#) qui retourne un objet de type PMap. Pour information, cet objet PMap est l'objet que l'on utilise dans cette classe pour savoir si la requête a des arguments « legacy ». On mock alors la méthode [has\(\)](#) pour qu'elle retourne comme valeur de base le booléen faux pour tout string (cas de base dans lequel on n'a aucun paramètre « legacy » dans la requête GHRequest).

On a donc cinq tests pour tester tous les chemins possibles de cette méthode :

- [testNoLegacyParameters](#) (le « happy path » dans lequel aucune exception n'est lancée parce qu'il n'y pas de paramètre « legacy »)
- [testVehicleParameter](#) (le chemin dans lequel une exception est lancée à cause du paramètre obsolète « vehicle »)
- [testWeightParameter](#) (le chemin dans lequel une exception est lancée à cause du paramètre obsolète « weighting »)
- [testTurnCostsParameter](#) (le chemin dans lequel une exception est lancée à cause du paramètre obsolète « TURN_COSTS »)
- [testEdgeBasedParameter](#) (le chemin dans lequel une exception est lancée à cause du paramètre obsolète « EDGE_BASED »)

```

134 void checkNoLegacyParameters(GHRequest request) {
135     if (request.getHints().has("vehicle"))
136         throw new IllegalArgumentException("GHRequest may no longer contain a vehicle, use the profile parameter instead, see docs/core/profiles.md");
137     if (request.getHints().has("weighting"))
138         throw new IllegalArgumentException("GHRequest may no longer contain a weighting, use the profile parameter instead, see docs/core/profiles.md");
139     if (request.getHints().has(Parameters.Routing.TURN_COSTS))
140         throw new IllegalArgumentException("GHRequest may no longer contain the turn_costs=true/false parameter, use the profile parameter instead, see docs/core/profiles.md");
141     if (request.getHints().has(Parameters.Routing.EDGE_BASED))
142         throw new IllegalArgumentException("GHRequest may no longer contain the edge_based=true/false parameter, use the profile parameter instead, see docs/core/profiles.md");
143 }

```

On peut voir que dans le rapport Pitest, tous les mutants de la méthode checkNoLegacyParameters ont été tués

En bref, on a fait le **mock de trois classes** distinctes pour les tests : Map, PMap et GHRequest.

Test du score de mutation

(Note 1 : On doit ajouter la [permission pour écrire](#) au début du build.yml afin que le score de mutation peut être stocké automatiquement s'il y a une amélioration du score de mutation.)

(Note 2 : Puisque l'on manipule un fichier dans lequel on stocke le score de mutation comme dit précédemment, on ne roule le script pour mettre à jour le score que sur une version de Java (deux workflows sont lancés à chaque fois pour la version de Java 24 et 25ea) pour éviter quelconque conflit.)

En bref, le but du [script pour le test de mutation](#) est de faire tourner Pitest, d'extraire le pourcentage calculé automatiquement par Pitest (qui provient du nombre de mutants tué sur tous les mutants créés) et de le stocker pour référence future.

En plus concret, voici ce qui se passe dans le script :

1. [Obtenir l'ancien score de mutation](#) stocké dans [mutation-threshold.txt](#) (qui ne contient qu'un nombre entier). Si ce fichier n'existe pas, [on le crée avec la valeur de base 0](#) qui est le score de mutation minimum dans lequel aucun mutant n'aurait été tué.
2. [Rouler Pitest](#). On a besoin ici de l'ancien score de mutation afin de définir un minimum pour lequel le build échouera s'il est en dessous de celui-ci. On utilise « tee » pour enregistrer les logs dans un fichier temporaire ([pitest_output.log](#)), mais aussi pour qu'ils soient affichés dans la console. Si l'étape échoue (si le « PIPESTATUS[0] » n'est pas égal à 0, ce qui signifie « succès » dans Github Actions), [on doit alors échouer le build](#). À noter ici que l'on utilise « -pl core » pour ne viser que le module « core » puisque dans notre cas, on n'utilise que Pitest avec ce module.
3. [Obtenir le nouveau score de mutation](#) à partir des logs ([pitest_output.log](#)). On cherche ici à [trouver la ligne](#) de type « Generated X mutations Killed Y (Z%) » dans laquelle X, Y et Z sont des entiers et Z est le score de mutation que l'on cherche à extraire. Si telle ligne n'existe pas, [le build échoue](#). On cherche par la suite à [extraire le score de mutation lui-même](#) (la partie Z) et le cas où tel score ne pourrait pas être extrait est géré similairement au cas précédent où la ligne n'existait pas dans les logs, c'est-à-dire en [échouant le build](#).

4. [Mettre à jour le score de mutation](#) s'il s'est amélioré. S'il n'a pas augmenté (l'ancien score de mutation est égal au nouveau), on ne change rien. S'il a augmenté, il faudra alors changer le [mutation-threshold.txt](#). [En premier](#), on devra mettre en place un « bot » (l'identifier en lui associant un nom et une adresse courriel). [Ensuite](#), il faudra préparer le nouveau [mutation-threshold.txt](#) mis à jour avec le nouveau score de mutation, ajouter le fichier à la liste des changements avec « add » et « commit » par la suite ce changement. [Finalement](#), il faudra « push » le « commit » vers la branche « master » afin que la version finale affichée sur Github contienne aussi le score de mutation mis à jour qui pourra être utilisé comme future référence.

chore(ci): Auto-update mutation threshold to 4%

6f4bede  <>

 github-actions[bot] committed 4 days ago

Exemple de quoi ressemble la mise à jour du score de mutation (point 4) dans l'historique de commits

Rickroll

Pour le [rickroll](#), on cherche à laisser un message que le développeur verra lorsque [l'étape précédente](#) dans laquelle on effectue les tests échoue. Pour ce faire, il faut dire au workflow de continuer de rouler même s'il y a une erreur (« continue-on-error: true ») pour permettre à l'étape du rickroll de se déclencher. Le rickroll est affiché sous forme d'erreur (du moins, il prend sa couleur et sa forme) avec un lien pour avoir « plus de détails » sur cette erreur qui nous amène à la [vidéo officielle elle-même de la chanson](#) :

```
▼ [x] Generate diagnostics for the failed test... 0s

1  ▶ Run echo " "
20
21  Error: A critical assertion failed. Analyzing stack trace...
22  Error:
23  Error: java.lang.AssertionError: Test expectations not met.
24  Error:   at com.graphhopper.Never.gonnaGiveYouUp(TestHelper.java:42)
25  Error:   at com.graphhopper.Never.gonnaLetYouDown(TestHelper.java:43)
26  Error:   at com.graphhopper.Never.gonnaRunAroundAndDesertYou(TestHelper.java:44)
27  Error:   at java.base/java.lang.reflect.Method.invoke(Method.java:568)
28  Error:   at org.junit.platform.engine.support.hierarchical.NodeTestTask.execute(NodeTestTask.java:75)
29  Error:   ... 23 more
30  Error:
31  Error: See full report for details: https://www.youtube.com/watch?v=dQw4w9WgXcQ
32  Error:
33  Error: Process completed with exit code 1.
```

Message affiché dans la section Actions de Github lorsqu'un test échoue

Puisque l'on a dit au « workflow » de rouler même s'il y a une erreur, il faut alors quitter manuellement (« [exit 1](#) »).